ESCUELA DE INGENIERÍA EN COMPUTACIÓN
INGENIERÍA EN DESARROLLO DE SOFTWARE

ING. GILBERT GARCÍA

EVALUACIÓN PRÁCTICA N.º 2 – IDS221

Gestión de Datos para Inteligencia Artificial y Big Data	
Integrantes	
1.	Gabriela Michelle García Pérez
2.	Cristina Raquel Alfaro Aguilar
3.	Isela Alexandra Quintanilla Cruz
4.	Glenda Beatriz Cruz García
5.	Luis Alonso Hernández Mundo
6.	Emerson Alexander Granadeño Valiente

INTEGRIDAD ACADÉMICA: Pueden consultar los materiales trabajados en clase y documentación técnica. Durante el examen no se permite delegar el desarrollo, el razonamiento ni las justificaciones a asistentes de IA generativa, terceros o soluciones ajenas. Las evidencias deben poder explicarse y reproducirse en una comprobación breve ante el docente. (Vídeo de 5 minutos máximo)

1. Competencia general

Implementar una solución analítica reproducible a partir de datos empresariales mediante visualización interactiva, análisis geoespacial, reportería automatizada y predicción básica para sustentar decisiones operativas.

2. Competencias a evaluar

Componente	Desempeño verificable	Guía
Dashboard de negocio	Construye medidas DAX, KPIs y segmentadores; comprueba resultados.	5
Análisis geoespacial	Valida coordenadas y configura capas y dos vistas territoriales.	6
Reportería automatizada	Regenera informe con KPIs, gráficos, plantilla y metadata.	7
Modelado predictivo	Compara baseline y regresión lineal con partición temporal y métricas.	8
Argumentación técnica	Defiende decisiones con evidencias reales de su ejecución.	Transversal

3. Caso de estudio: GreenEnergy Analytics

GreenEnergy Analytics continúa modernizando su operación energética en El Salvador. Después de integrar, limpiar y proteger datos (Evaluación Práctica 1), la gerencia requiere un tablero de cobertura energética, una lectura territorial de sus sedes, un informe que pueda actualizarse y un pronóstico básico de demanda diaria. El equipo consultor dispone de 180 días de operación sintética de nueve sedes, con una ubicación registrada incorrectamente. La empresa necesita decisiones sustentadas; no le sirve una presentación estética sin cifras verificables.

ROL: Ustedes actúan como equipo de Ingeniería en Desarrollo de Software contratado para entregar, en 24 horas, un prototipo analítico funcional y técnicamente defendible. Se trabajará con un solo conjunto de datos y los mismos requerimientos para todos los grupos.

4. Fuentes entregadas por la empresa

Archivo	Campos esenciales	Uso
operacion_sedes.csv	fecha, sede_id, region, generacion_kwh, demanda_kwh, evento_programado	Power BI, acumulación territorial y reporte
sedes_geograficas.csv	sede_id, nombre_sede, municipio, region, latitud, longitud	Mapa ArcGIS; incluye un registro para depurar
demanda_diaria.csv	fecha, demanda_kwh, evento_programado	Baseline, regresión lineal y evaluación temporal

5. Ruta sugerida de las 3 horas (sugerido)

Momento	Minutos	Producto
Lectura, roles y archivos	0–10	Equipo y fuentes verificados
Parte 1: Power BI	10–45	Dashboard + indicadores.csv
Parte 2: ArcGIS	45–75	CSV geográfico + 2 vistas
Parte 3: Reportería	75–115	Informe automatizado + metadata
Parte 4: Predicción	115–165	Predicciones + métricas
Integración y verificación	165–180	ZIP final, justificaciones, comprobación rápida

Se permite paralelizar tareas por integrante; sin embargo, todos deben comprender al menos una decisión y un resultado cuantitativo del producto global. Se debe consolidar en un solo entregable de grupo (Zip, repositorio y video)

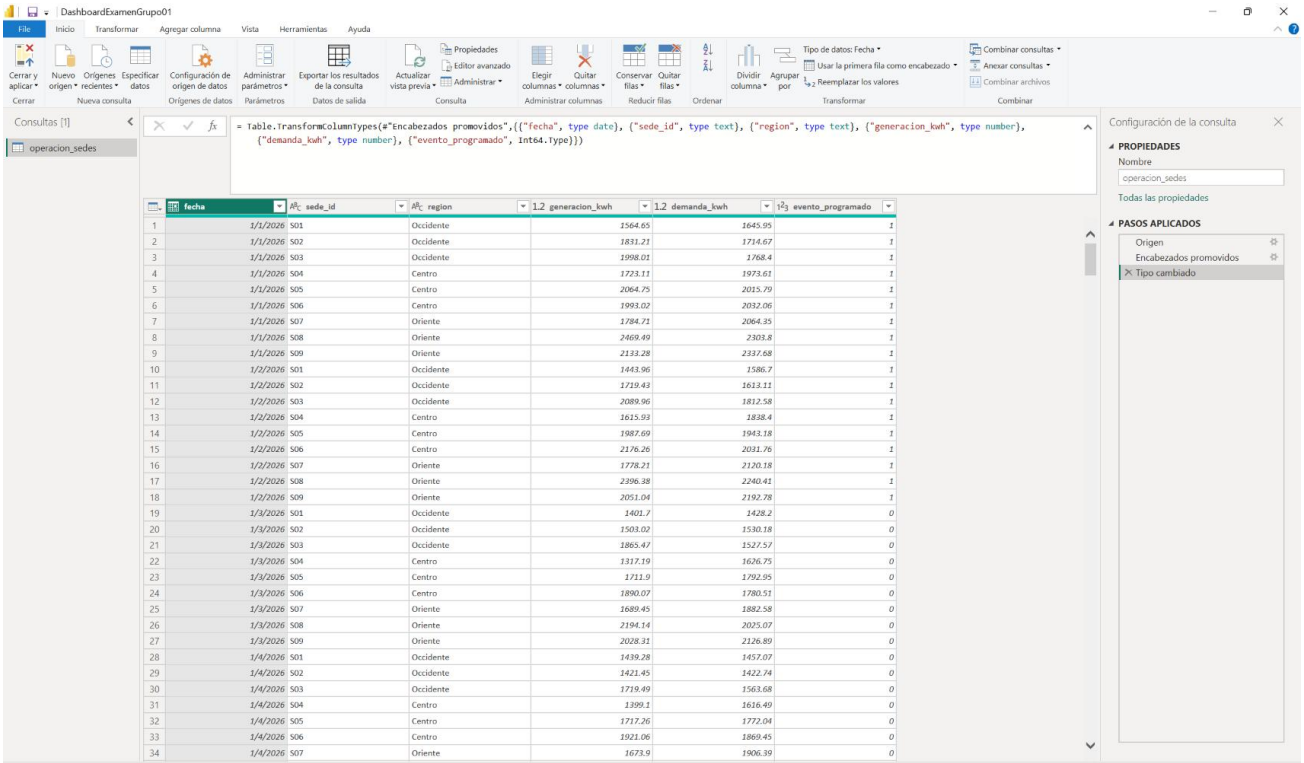
6. Desarrollo de la evaluación

Se debe colocar captura de pantalla que evidencie cada paso

Parte 1. Dashboard interactivo con Power BI — 25 puntos

Problema: La gerencia desconoce qué proporción de la demanda queda cubierta con generación propia y en qué sedes se concentra el déficit.

- Importar operacion_sedes.csv en Power BI Desktop, comprobar tipos de fecha y datos numéricos.

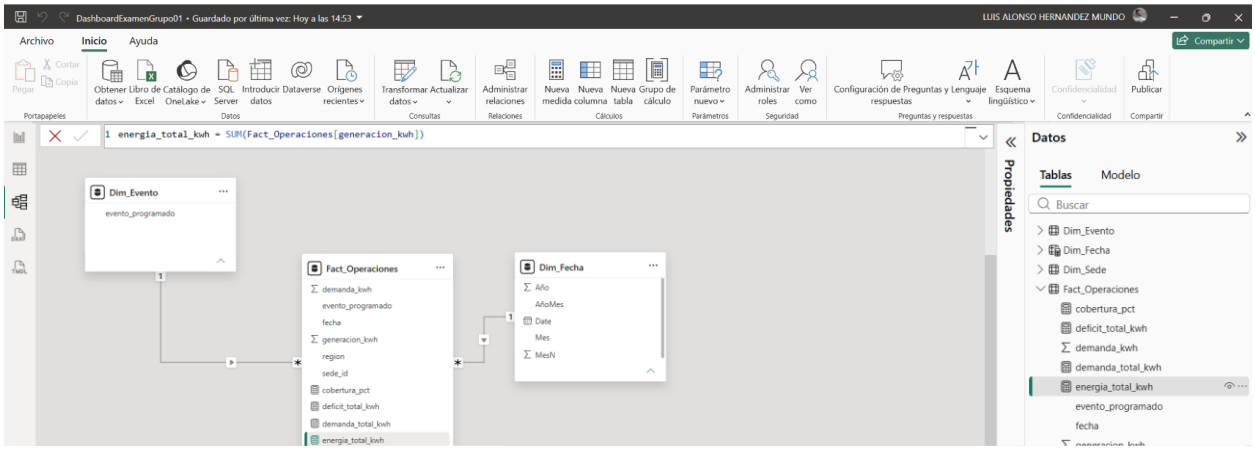


	fecha	sede_id	region	generacion_kwh	demanda_kwh	evento_programado
1	1/1/2026	S01	Occidente	1564.65	1645.95	1
2	1/1/2026	S02	Occidente	1831.21	1714.67	1
3	1/1/2026	S03	Occidente	1998.01	1768.4	1
4	1/1/2026	S04	Centro	1723.11	1973.61	1
5	1/1/2026	S05	Centro	2064.75	2015.79	1
6	1/1/2026	S06	Centro	1993.02	2032.06	1
7	1/1/2026	S07	Oriente	1784.71	2064.35	1
8	1/1/2026	S08	Oriente	2469.49	2303.8	1
9	1/1/2026	S09	Oriente	2133.28	2337.68	1
10	1/2/2026	S01	Occidente	1443.96	1586.7	1
11	1/2/2026	S02	Occidente	1719.43	1613.11	1
12	1/2/2026	S03	Occidente	2089.96	1812.58	1
13	1/2/2026	S04	Centro	1615.93	1838.4	1
14	1/2/2026	S05	Centro	1987.69	1943.18	1
15	1/2/2026	S06	Centro	2176.26	2031.76	1
16	1/2/2026	S07	Oriente	1778.21	2120.18	1
17	1/2/2026	S08	Oriente	2396.38	2240.41	1
18	1/2/2026	S09	Oriente	2051.04	2192.78	1
19	1/3/2026	S01	Occidente	1401.7	1428.2	0
20	1/3/2026	S02	Occidente	1503.02	1530.18	0
21	1/3/2026	S03	Occidente	1865.47	1527.57	0
22	1/3/2026	S04	Centro	1317.19	1626.75	0
23	1/3/2026	S05	Centro	1711.9	1792.95	0
24	1/3/2026	S06	Centro	1890.07	1780.51	0
25	1/3/2026	S07	Oriente	1689.45	1882.58	0
26	1/3/2026	S08	Oriente	2194.14	2025.07	0
27	1/3/2026	S09	Oriente	2028.31	2126.89	0
28	1/4/2026	S01	Occidente	1439.28	1457.07	0
29	1/4/2026	S02	Occidente	1421.45	1422.74	0
30	1/4/2026	S03	Occidente	1719.49	1563.68	0
31	1/4/2026	S04	Centro	1399.1	1616.49	0
32	1/4/2026	S05	Centro	1717.26	1772.04	0
33	1/4/2026	S06	Centro	1921.06	1869.45	0
34	1/4/2026	S07	Oriente	1673.9	1906.39	0

- Construir cuatro medidas DAX:

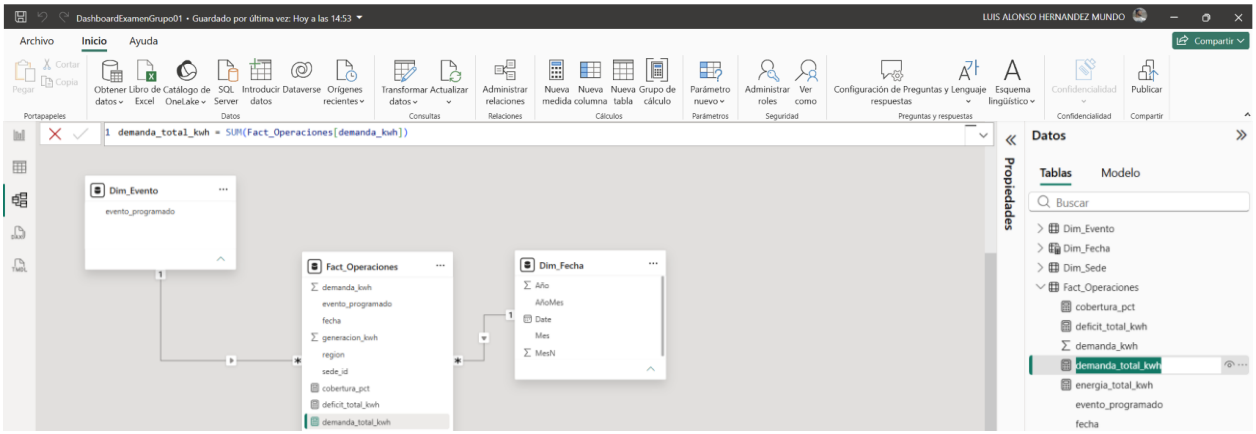
1. Energía generada total.

Medida DAX usada: energia_total_kwh = SUM(Fact_Operaciones[generacion_kwh])



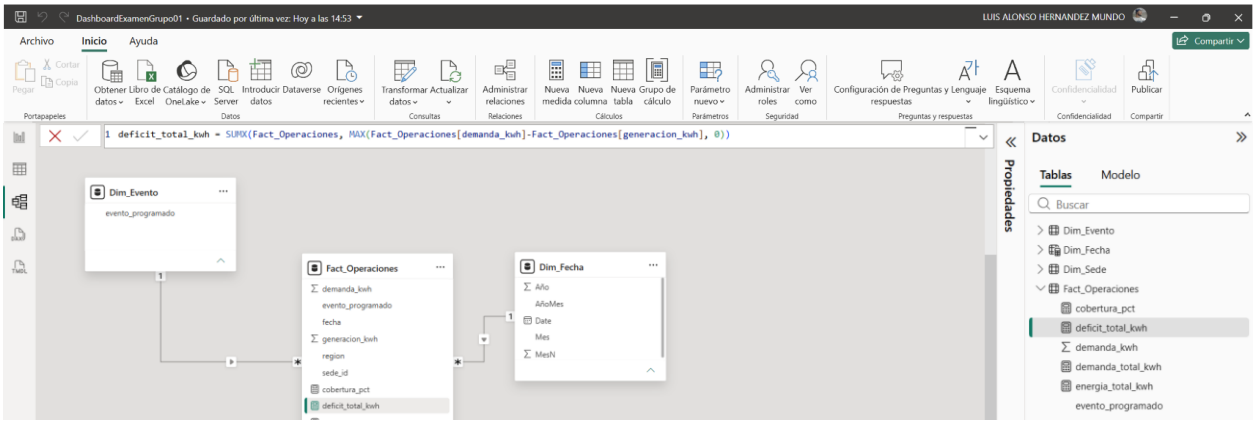
1. Demanda total.

Medida DAX usada: demanda_total_kwh = SUM(Fact_Operaciones[demanda_kwh])



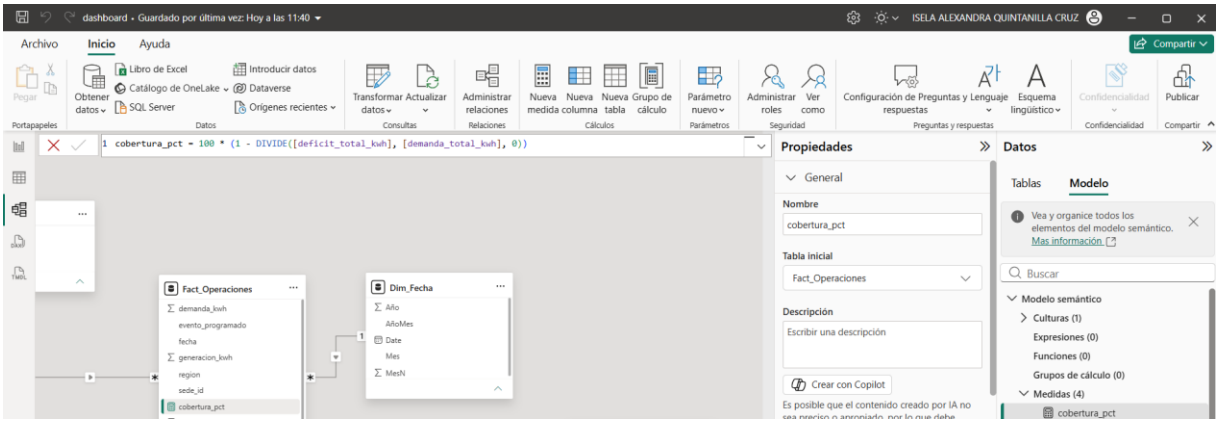
2. Déficit energético total (suma de MAX(demanda – generación, 0) por registro).

Medida DAX usada: deficit_total_kwh = SUMX(Fact_Operaciones, MAX(Fact_Operaciones[demanda_kwh]-Fact_Operaciones[generacion_kwh], 0))



3. Cobertura (%) = 100 × (1 − déficit total / demanda total).

- Medida DAX usada: cobertura_pct = 100 * (1 - DIVIDE([deficit_total_kwh], [demanda_total_kwh], 0))



- Diseñar una página con las cuatro tarjetas KPI, un gráfico de evolución temporal y gráficos comparativos por región y sede; incorporar segmentador por región.

Tarjetas:

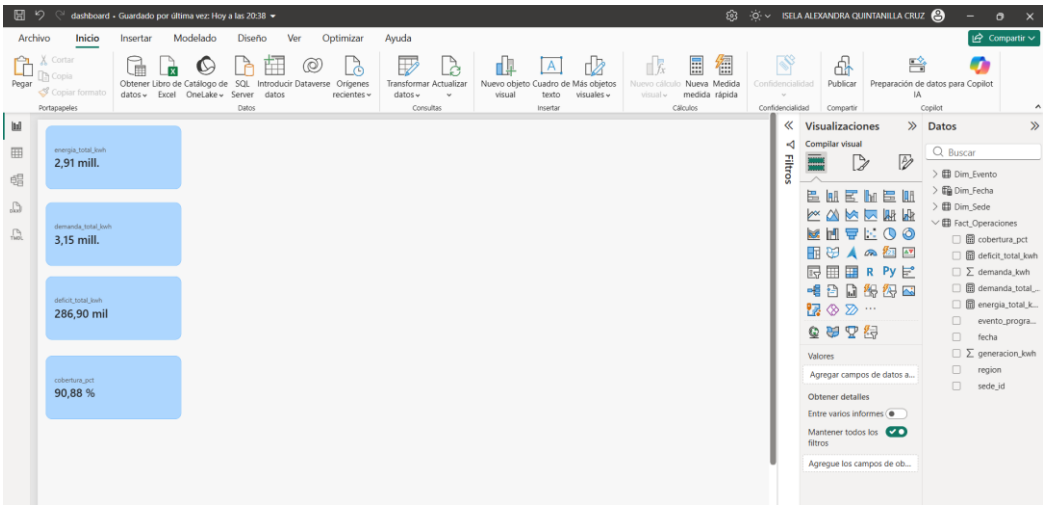
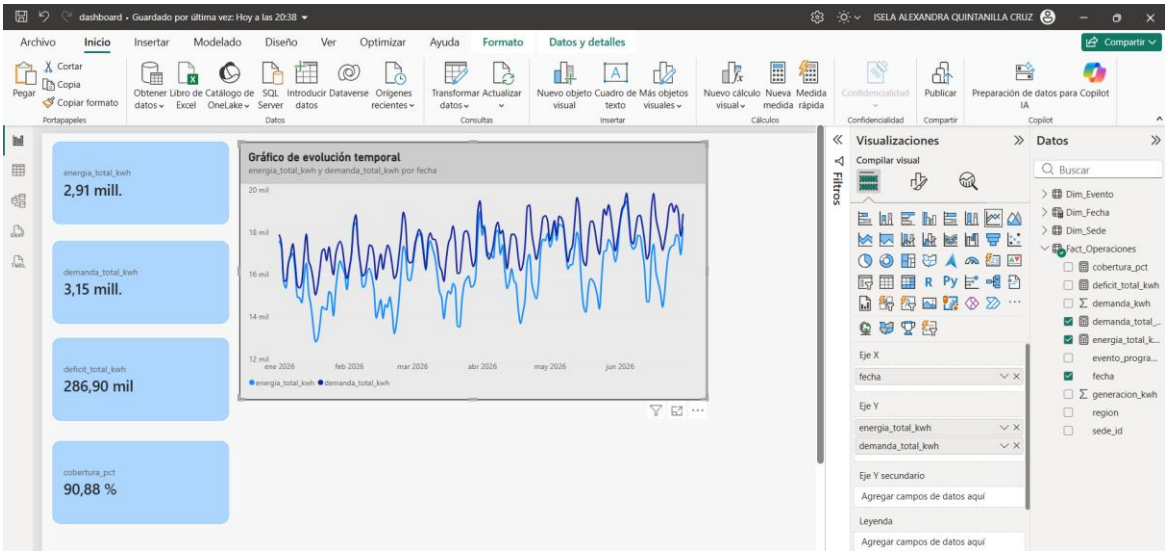
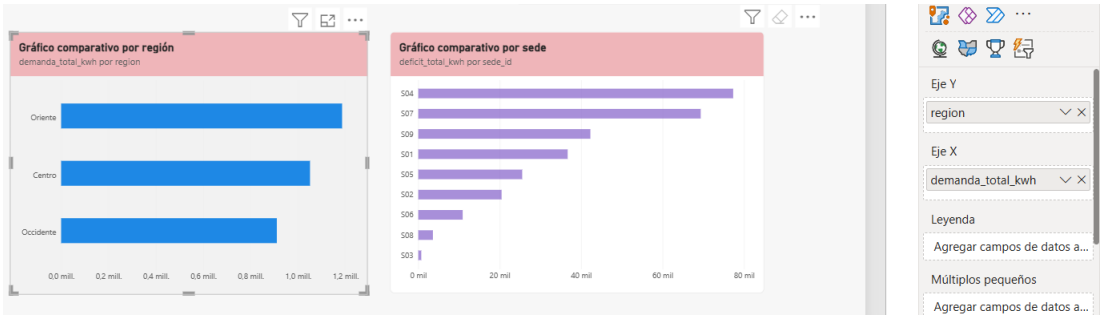


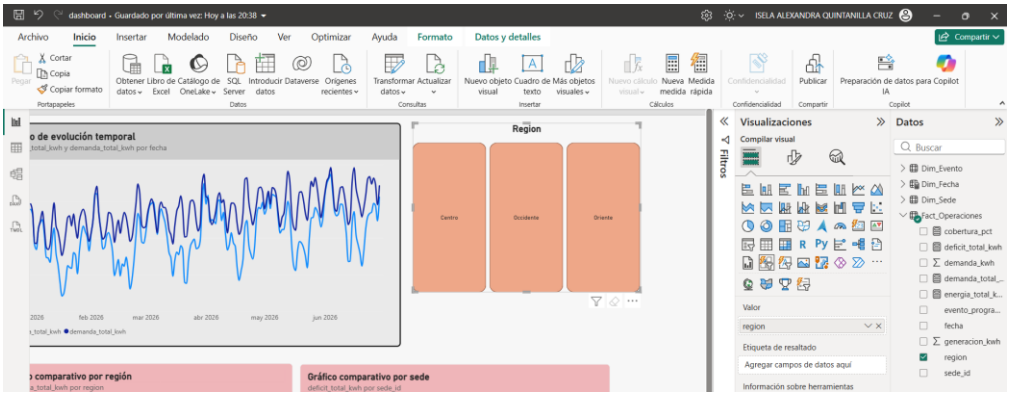
Gráfico de evolución temporal:



Gráficos comparativos por región y sede:



Segmentación por botones:

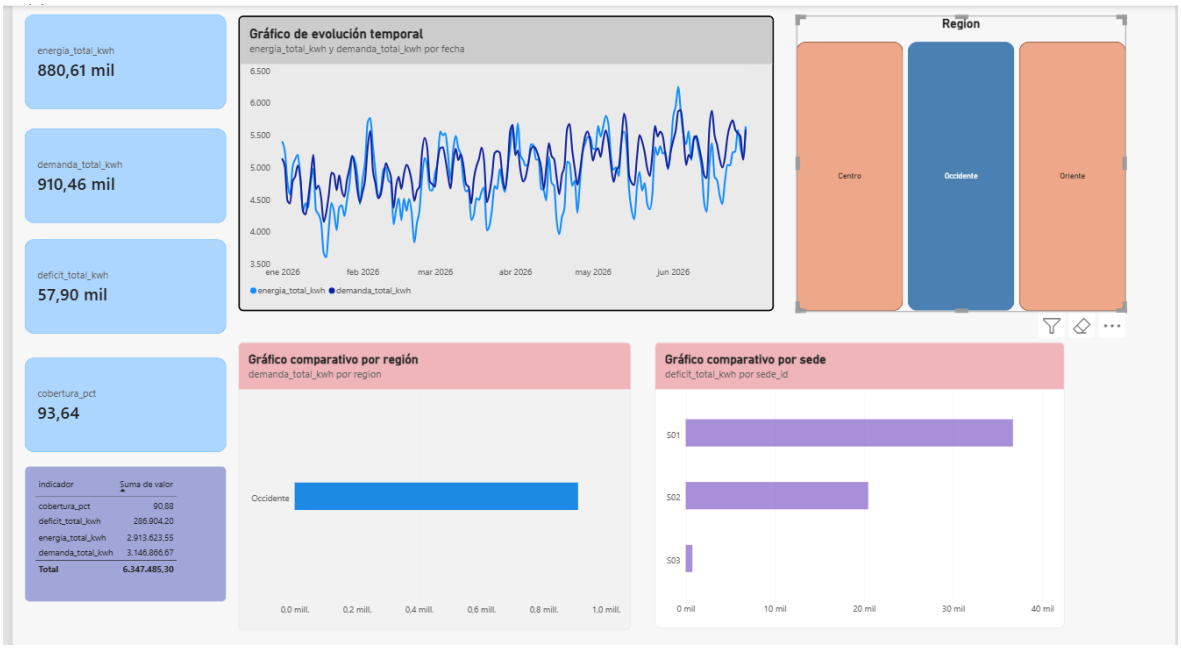


- Verificar una interacción: seleccionar una región y comprobar que cambian los indicadores; **guardar dashboard.pbix y vista_bi.png.**

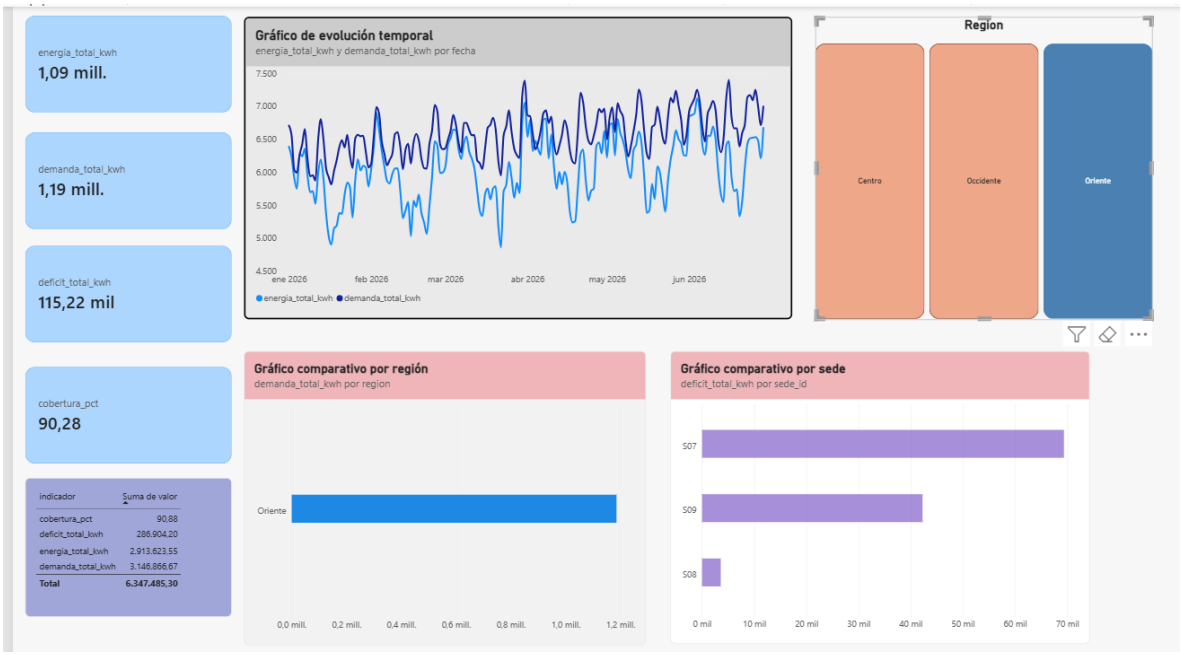
Seleccionar Centro:

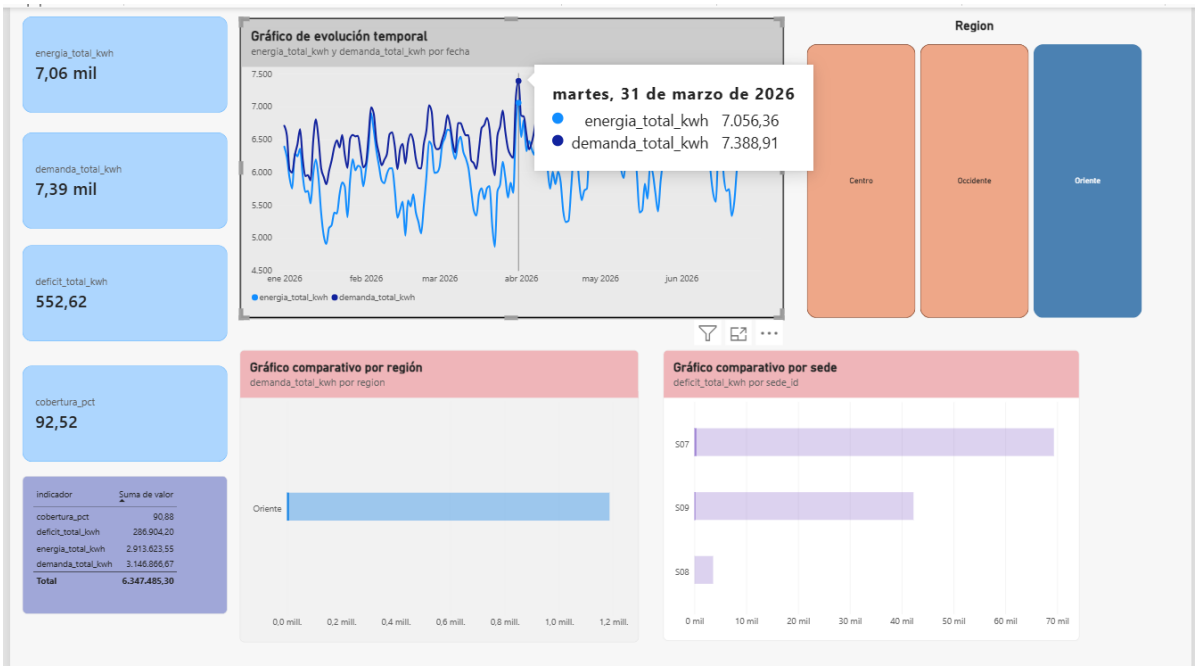


Seleccionar Occidente:



Seleccionar Oriente:





- Entregar indicadores.csv con columnas indicador,valor; usar exactamente las cuatro claves del contrato de salida. Valores numéricos sin unidades y con punto decimal.

Justificación técnica (máximo 5 líneas): cite el déficit total y la cobertura % antes de filtrar. Explique qué medida no puede obtenerse sumando déficits netos globales y por qué el segmentador altera el contexto de cálculo.

Respuesta:

Antes de filtrar con el segmentador, el déficit total es de **286,90 mil kWh** y la cobertura porcentual es de **90.88 %.** La medida que no puede obtenerse sumando déficits netos globales es el déficit energético total. Ya que el restar los totales (demanda_total_kwh – energía_total_kwh = 233.243,12 kwh) produce un déficit neto donde el excedente de unas sedes tapa el faltante de otras; siendo el déficit real 286,90 mil kwh. El segmentador altera el contexto del cálculo porque filtra las filas de las tablas antes de que las medidas DAX se apliquen, cuando se selecciona una sola región, las medidas ya no suman las 9 sedes, sino solamente las sedes de esa región, y se recalculan los KPIs solo para la región seleccionada.

Parte 2. Análisis geoespacial con ArcGIS — 20 puntos

Problema: La gerencia debe localizar sedes con déficit energético y detectar errores de ubicación antes de comunicar un mapa.

```python
#PASO1: CREACION DE LA ESTRUCTURA
import pandas as pd
from pathlib import Path
import numpy as np

BASE = Path('/content/proyecto_Arc_Gis')

carpetas = [
    'datos/geo',
    'documentos/geo',
    'evidencias/geo',
    'visualizaciones',
    'notebooks',
]

for c in carpetas:
  (BASE / c).mkdir(parents=True, exist_ok=True)

(BASE / 'README.md').touch()

print("ESTRUCURAS CREADAS:----------------------")
for p in sorted(BASE.rglob('*')):
  nivel = len(p.relative_to(BASE).parts) - 1
  print('  ' * nivel + p.name + ('/' if p.is_dir() else ''))
```

```python
#PASO2: DEFINIR LAS RUTAS EN LAS QUE ESTARAN LOS CSV QUE TRABAJAREMOS
#DEFINIR RUTA DEL CSV DE DEMANDA_DIARIA

ruta_dem = BASE / 'datos' / 'geo' / 'demanda_diaria.csv'

#DEFINIR RUTA DEL CSV DE OPERACION_SEDES

ruta_ope = BASE / 'datos' / 'geo' / 'operacion_sedes.csv'

#DEFINIR LA RUTA DEL CSV DE SEDES_GEOGRAFICAS

ruta_geo = BASE / 'datos' / 'geo' / 'sedes_geograficas.csv'

faltantes = [r for r in (ruta_dem, ruta_ope, ruta_geo) if not r.exists()]
if faltantes:
  raise FileNotFoundError(f"Faltan: {faltantes}")
else:
  print("Ambos archivos CSV existen y estan listos para usar]")
```

```
Ambos archivos CSV existen y estan listos para usar]
```

```
#PASO3: LEER LOS CSV QUE TENEMOS A LA MANO
#LEER CSV DE DEMANDA_DIARIA

df_dem = pd.read_csv(ruta_dem)

#LEER CSV DE OPERACION_SEDES

df_ope = pd.read_csv(ruta_ope)

#LEER CSV DE SEDES_GEOGRAFICAS

df_geo = pd.read_csv(ruta_geo)

#IMPRIMIR INFORMACION DE DEMANDA_DIARIA

print("\n----------Archivo Demanda_diaria----------\n")
print(f"Registros: {len(df_dem)}")
print(f"Columnas: {df_dem.columns.tolist()}")
print(df_dem.head())

#IMPRIMIR INFORMACION DE OPERACIONES_SEDES

print("\n----------Archivo Operaciones_Sedes----------\n")
print(f"Registros: {len(df_ope)}")
print(f"Columnas: {df_ope.columns.tolist()}")
print(df_ope.head())

#IMPRIMIR INFORMACION DE SEDES_GEOGRAFICOS
```

```
#IMPRIMIR INFORMACION DE SEDES_GEOGRAFICOS

print("\n----------Archivo Sedes_Geograficos----------\n")
print(f"Registros: {len(df_geo)}")
print(f"Columnas: {df_geo.columns.tolist()}")
print(df_geo.head())
```

```
----------Archivo Demanda_diaria----------

Registros: 180
Columnas: ['fecha', 'demanda_kwh', 'evento_programado']
        fecha  demanda_kwh  evento_programado
0  2026-01-01     17856.31                  1
1  2026-01-02     17379.10                  1
2  2026-01-03     15720.70                  0
3  2026-01-04     15694.05                  0
4  2026-01-05     16505.77                  0

----------Archivo Operaciones_Sedes----------

Registros: 1620
Columnas: ['fecha', 'sede_id', 'region', 'generacion_kwh', 'demanda_kwh', 'evento_programado']
        fecha sede_id     region  generacion_kwh  demanda_kwh  \
0  2026-01-01     S01  Occidente         1564.65      1645.95
1  2026-01-01     S02  Occidente         1831.21      1714.67
2  2026-01-01     S03  Occidente         1998.01      1768.40
3  2026-01-01     S04     Centro         1723.11      1973.61
4  2026-01-01     S05     Centro         2064.75      2015.79

   evento_programado
```

```
#PASO4: SUMA DE GLOBAL DE DEFICIT, DEMANDA, Y GENERACION
print("----------------METRICAS GLOBALES DEL SISTEMA----------------")
total_gen_global = df_ope['generacion_kwh'].sum()
total_dem_global = df_ope['demanda_kwh'].sum()

deficit_dashboard = np.maximum(df_ope['demanda_kwh'] - df_ope['generacion_kwh'], 0).sum()

print(f"Energía generada: {total_gen_global:,.2f} kWh")
print(f"Demanda total:    {total_dem_global:,.2f} kWh")
print(f"Déficit total:    {deficit_dashboard:,.2f} kWh")
```

```
----------------METRICAS GLOBALES DEL SISTEMA----------------
Energía generada: 2,913,623.55 kWh
Demanda total:    3,146,866.67 kWh
Déficit total:    286,904.20 kWh
```

- Convertir coordenadas a numéricas y verificar plausibilidad territorial: 13 a 15 para latitud y −91 a −87 para longitud. Excluir del mapa los registros inválidos sin inventar coordenadas.

```
#PASO5: CONVERTIR COORDENADAS A NUMERO PARA SEDES

df_geo['latitud'] = pd.to_numeric(df_geo['latitud'], errors='coerce')
df_geo['longitud'] = pd.to_numeric(df_geo['longitud'], errors='coerce')

print("Coordenadas convertidas a numerico----------------------")
print("----------Sedes Geograficas------------")
print(df_geo[['municipio', 'latitud', 'longitud']])
```

```
Coordenadas convertidas a numerico----------------------
----------Sedes Geograficas------------
      municipio  latitud  longitud
0    Ahuachapán  13.9214  -89.8453
1     Santa Ana  13.9942  -89.5597
2     Sonsonate  13.7189  -89.7241
3   San Salvador  13.6929  -89.2182
4    Santa Tecla  13.6769  -89.2797
5   Cojutepeque  13.7179  -88.9331
6    San Miguel  13.4837  -88.1779
7      Usulután  13.3500  -88.4461
8      La Unión  13.3369  -67.8440
```

```python
#PASO6: VALIDACION DE PLAUSBILIDAD PARA EL SALVADOR\
mask_valida = (
    df_geo['latitud'].between(13, 15) &
    df_geo['longitud'].between(-91, -87)
)

print(f"Total de registros: {len(df_geo)}")
print(f"Coordenadas validas: {mask_valida.sum()}")
print(f"Coordenada a revisar: {(~mask_valida).sum()}" )


if (~mask_valida).sum() > 0:
  print("\n Coordenadas invalidas: ---------------")
  print(df_geo[~mask_valida])
else:
  print("\nTodas las coordanas estan dentro de El Salvador ------------------------")
```

```
Total de registros: 9
Coordenadas validas: 8
Coordenada a revisar: 1

 Coordenadas invalidas: ---------------
  sede_id     nombre_sede municipio   region  latitud  longitud
8     S09  Solar La Unión  La Unión  Oriente  13.3369   -67.844
```

- Calcular con Python/Pandas el déficit acumulado por sede desde operacion_sedes.csv y asociarlo por sede_id con sedes_geograficas.csv.

```python
#PASO7: CALCULAR EL DEFICIT POR SEDE APLICANDO LA LÓGICA DAX (IGNORANDO EXCEDENTES)

df_ope['deficit_positivo'] = np.maximum(df_ope['demanda_kwh'] - df_ope['generacion_kwh'], 0)


df_deficit_sede = df_ope.groupby('sede_id', as_index=False)['deficit_positivo'].sum()
df_deficit_sede.rename(columns={'deficit_positivo': 'deficit_total_kwh'}, inplace=True)


df_geo_merged = pd.merge(df_geo, df_deficit_sede, on='sede_id', how='left')

print("--- Déficit acumulado por sede  ---")
print(df_geo_merged[['sede_id', 'nombre_sede', 'deficit_total_kwh']].sort_values(by='deficit_total_kwh', ascending=False).reset_index(drop=True))
```

```
--- Déficit acumulado por sede  ---
  sede_id         nombre_sede  deficit_total_kwh
0     S04  Solar San Salvador           77336.41
1     S07    Solar San Miguel           69364.49
2     S09      Solar La Unión           42266.81
3     S01    Solar Ahuachapán           36692.27
4     S05   Solar Santa Tecla           25528.84
5     S02     Solar Santa Ana           20465.48
6     S06   Solar Cojutepeque           10909.28
7     S08      Solar Usulután            3593.47
8     S03     Solar Sonsonate             747.15
```

- Exportar geo_validado.csv con las columnas sede_id,latitud,longitud,deficit_total_kwh. Crear la capa de puntos en ArcGIS Map Viewer.

```python
#PASO8: FILTRAR Y ELIMINAR LA SEDE S09 DEL CONJUNTO FINAL
df_geo_validado = df_geo_merged[mask_valida].copy()

df_final_arcgis = (
    df_geo_validado[['sede_id', 'latitud', 'longitud', 'deficit_total_kwh']]
    .sort_values(by='deficit_total_kwh', ascending=False)
    .reset_index(drop=True)
)


df_final_arcgis['deficit_total_kwh'] = df_final_arcgis['deficit_total_kwh'].round(2)

print("--- Archivo 'geo_validado.csv' generado correctamente ---")
print(df_final_arcgis)
```

```
--- Archivo 'geo_validado.csv' generado correctamente ---
  sede_id  latitud  longitud  deficit_total_kwh
0     S04  13.6929  -89.2182           77336.41
1     S07  13.4837  -88.1779           69364.49
2     S01  13.9214  -89.8453           36692.27
3     S05  13.6769  -89.2797           25528.84
4     S02  13.9942  -89.5597           20465.48
5     S06  13.7179  -88.9331           10909.28
6     S08  13.3500  -88.4461            3593.47
7     S03  13.7189  -89.7241             747.15
```

```
#PASO9: EXPORTACION DE DATOS

#EXPORTAR VERSION VALIDADA

ruta_salida = BASE / 'datos' / 'geo' / 'geo_validado.csv'

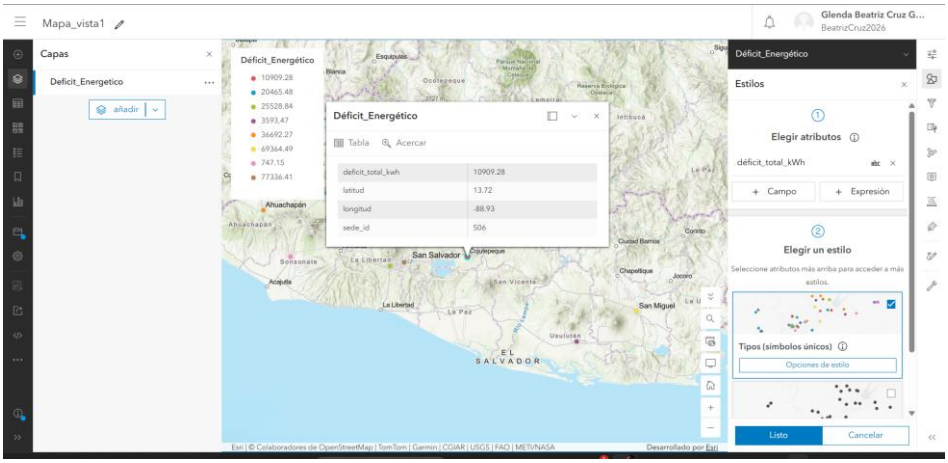
df_final_arcgis.to_csv(ruta_salida, index=False, encoding='utf-8')

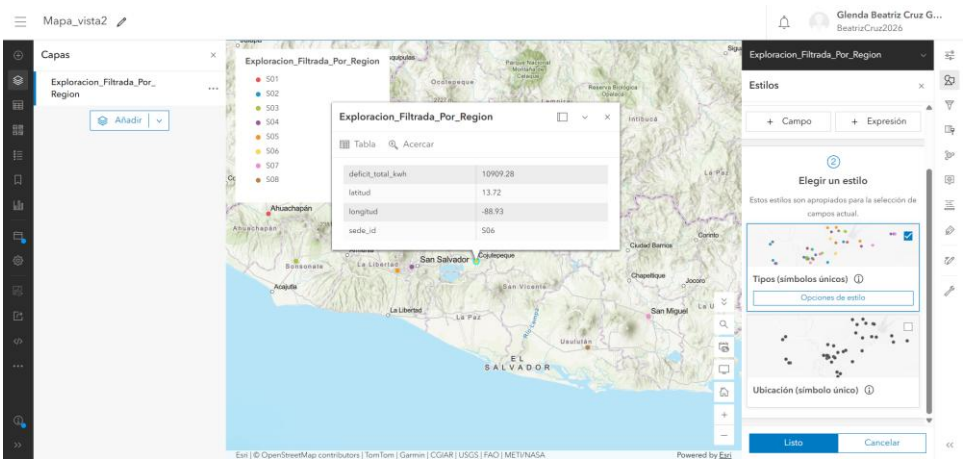
print(f"Archivo validado guardado: {ruta_salida}")

#DESCARGAR
from google.colab import files
files.download(str(ruta_salida))
```

```
Archivo validado guardado: /content/proyecto_Arc_Gis/datos/geo/geo_validado.csv
```

- Mostrar dos vistas del mismo análisis: una simbología por magnitud del déficit y otra exploración filtrada por región o estilizada por categoría; configurar pop-up informativo y capturar mapa_vista1.png y mapa_vista2.png.





- Registrar dos hallazgos territoriales concretos, distinguiendo observación de explicación causal.

```
#PASO6: VALIDACION DE PLAUSBILIDAD PARA EL SALVADOR\
mask_valida = (
    df_geo['latitud'].between(13, 15) &
    df_geo['longitud'].between(-91, -87)
)

print(f"Total de registros: {len(df_geo)}")
print(f"Coordenadas validas: {mask_valida.sum()}")
print(f"Coordenada a revisar: {(~mask_valida).sum()}" )


if (~mask_valida).sum() > 0:
  print("\n Coordenadas invalidas: ---------------")
  print(df_geo[~mask_valida])
else:
  print("\nTodas las coordanas estan dentro de El Salvador ----------------------")
```

```
Total de registros: 9
Coordenadas validas: 8
Coordenada a revisar: 1

 Coordenadas invalidas: ---------------
  sede_id     nombre_sede municipio   region  latitud  longitud
8     S09  Solar La Unión  La Unión  Oriente  13.3369   -67.844
```

(El deficit_total_kwh se ubica de mayor a menor)

```
--- Déficit acumulado por sede  ---
  sede_id         nombre_sede  deficit_total_kwh
0     S04  Solar San Salvador           77336.41
1     S07    Solar San Miguel           69364.49
2     S09      Solar La Unión           42266.81
3     S01    Solar Ahuachapán           36692.27
4     S05   Solar Santa Tecla           25528.84
5     S02     Solar Santa Ana           20465.48
6     S06   Solar Cojutepeque           10909.28
7     S08      Solar Usulután            3593.47
8     S03     Solar Sonsonate             747.15
```

Justificación técnica (máximo 5 líneas): indique cuántas sedes recibieron validación, cuántas se excluyeron y qué sede presenta el mayor déficit válido. Explique el riesgo de mostrar coordenadas erróneas.

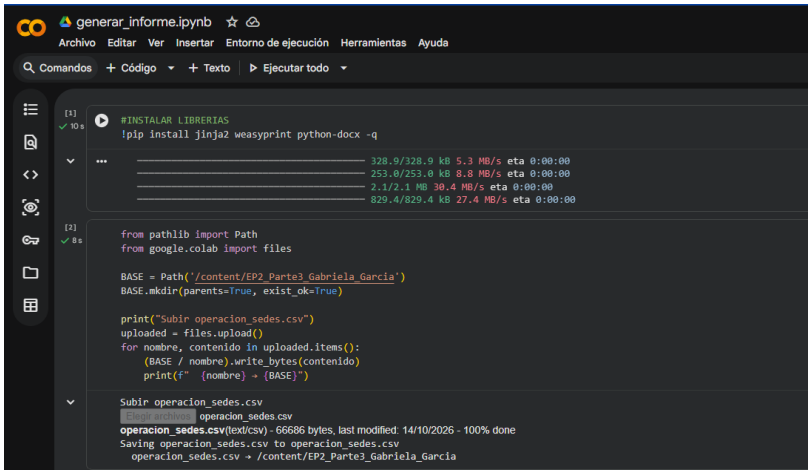
Respuesta:

De un total de 9 sedes, se validaron 8 sedes de los cuales cumplieron los criterios establecidos de Latitud y Longitud (latitud 13 a 15, longitud –91 a –87), excluyendo la sede con sede_id S09 la cual no cumplía la Longitud establecida (longitud –67). La sede con mayor déficit válido es la sede S04 (Solar San Salvador) el registro de déficit válido es de 77336.41 kWh. Mostrar coordenadas erróneas en mapas conlleva a un gran riesgo, comprometiendo la planeación operativa, la asignación de recursos y la precisión del análisis territorial el cual se basa el proyecto.

Parte 3. Reportería gerencial automatizada — 20 puntos

Problema: Cada semana se solicita un informe actualizado. No se permite copiar y pegar manualmente cifras que podrían variar.

- Crear generar_reporte.py que cargue el archivo de operaciones y calcule con Pandas energía generada, demanda y déficit totales.




```
# CARGAR DATASET
ruta_fuente = BASE/'operacion_sedes.csv'
df = pd.read_csv(ruta_fuente)
df['fecha'] = pd.to_datetime(df['fecha'])

print(f"Registros: {df.shape[0]} filas x {df.shape[1]} columnas")
print(f"Periodo: {df['fecha'].min().date()} a {df['fecha'].max().date()}") #esto es para verificar el periodo de tiempo en que se encuentra el archivo
print(df.head(3).to_string())
```

```
Registros: 1620 filas x 6 columnas
Periodo: 2026-01-01 a 2026-06-29
       fecha sede_id     region  generacion_kwh  demanda_kwh  evento_programado
0 2026-01-01     S01  Occidente         1564.65      1645.95                  1
1 2026-01-01     S02  Occidente         1831.21      1714.67                  1
2 2026-01-01     S03  Occidente         1998.01      1768.40                  1
```

```
# CALULAR LOS KPIS
# CALCULAMOS EL DEFICIT DE ENERGIA EN CADA SEDE
df['deficit_kwh'] = (df['demanda_kwh']-df['generacion_kwh']).clip(lower=0)
# utilizo .clip para que se vuelvan cero los negativos sugiriendonos que no hubo
# deficit de energia

kpis = {
    'energia_total_kwh': df['generacion_kwh'].sum(),
    'demanda_total_kwh': df['demanda_kwh'].sum(),
    'deficit_total_kwh': df['deficit_kwh'].sum(),
    'periodo': f"{df['fecha'].min().strftime('%d/%m/%Y')} - {df['fecha'].max().strftime('%d/%m/%Y')}",
}

print("KPIs DEL PERIODO")
print("=" * 50)
print(f"Energía generada:  {kpis['energia_total_kwh']:>14,.2f} kwh")
print(f"Demanda total:     {kpis['demanda_total_kwh']:>14,.2f} kwh")
print(f"Déficit total:     {kpis['deficit_total_kwh']:>14,.2f} kwh")
print(f"Periodo:           {kpis['periodo']}")
print("=" * 50)
```

```
KPIs DEL PERIODO
==================================================
Energía generada:    2,913,623.55 kwh
Demanda total:       3,146,866.67 kwh
Déficit total:         286,904.20 kwh
Periodo:           01/01/2026 - 29/06/2026
--------------------------------------------------
```

- Generar un gráfico en PNG con datos del caso, una plantilla Jinja2 llamada plantilla.html y su salida informe.html.

```
# ======================================================================
# GRÁFICOS PARA EL INFORME
# ======================================================================

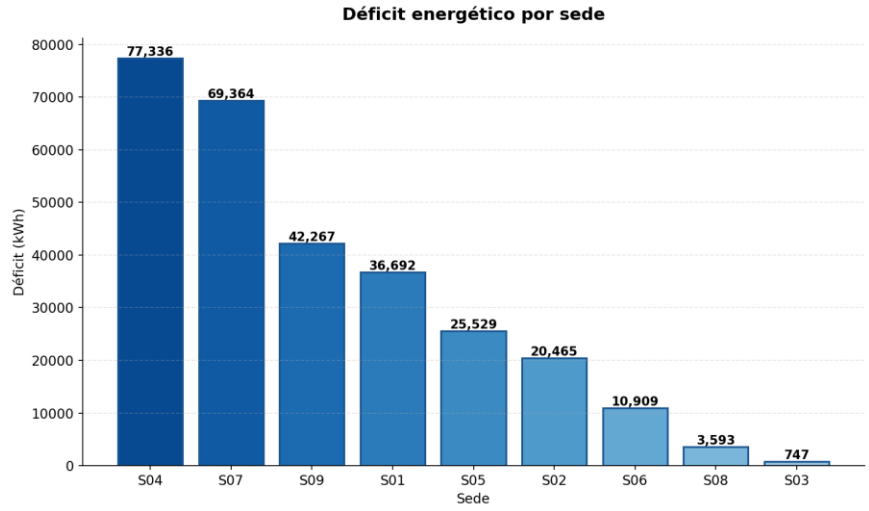
# GRÁFICO 1: Déficit por sede
fig, ax = plt.subplots(figsize=(10, 6))
colores = plt.cm.Blues(np.linspace(0.9, 0.4, len(df_sede)))   # más oscuro = más déficit

# Barras: eje X = sede, eje Y = déficit
barras = ax.bar(df_sede['sede_id'], df_sede['deficit_kwh'],
                color=colores, edgecolor='#1a5490', linewidth=1.5)

ax.set_title('Déficit energético por sede', fontsize=14, fontweight='bold', pad=15)
ax.set_xlabel('Sede', fontsize=11)
ax.set_ylabel('Déficit (kWh)', fontsize=11)
ax.grid(True, alpha=0.3, axis='y', linestyle='--')

for barra, valor in zip(barras, df_sede['deficit_kwh']):
    height = barra.get_height()
    ax.text(barra.get_x() + barra.get_width()/2., height,
            f'{valor:,.0f}', ha='center', va='bottom',
            fontsize=10, fontweight='bold')

plt.tight_layout()
ruta_grafico = BASE / 'grafico.png'
plt.savefig(ruta_grafico, dpi=150, bbox_inches='tight', facecolor='white')
plt.close()
print(f"Gráfico 1 (informe): {ruta_grafico.name}")
```




```
# ======================================================================
# PLANTILLA JINJA2 (Guía 07: "Template HTML con Jinja2")
# ======================================================================
template_informe = """
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Informe Gerencial - GreenEnergy Analytics</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; color: #333; line-height: 1.6; }
        h1 { color: #1a5490; border-bottom: 3px solid #1a5490; padding-bottom: 10px; }
        h2 { color: #2c7bb6; margin-top: 30px; border-bottom: 1px solid #ddd; padding-bottom: 5px; }
        .kpi-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 15px; margin: 20px 0; }
        .kpi-card { background: #f0f7ff; padding: 15px; border-left: 4px solid #2c7bb6; border-radius: 4px; }
        .kpi-valor { font-size: 22px; font-weight: bold; color: #1a5490; }
        .kpi-label { font-size: 12px; color: #666; text-transform: uppercase; }
        .hallazgo { background: #f9f9f9; padding: 15px; border-left: 4px solid #ff9800; margin: 15px 0; }
        .limitacion { background: #fff3cd; padding: 10px; border-left: 4px solid #ffc107; margin: 10px 0; }
        img { max-width: 100%; border: 1px solid #eee; }
        .pie { font-size: 11px; color: #999; margin-top: 40px; border-top: 1px solid #ddd; padding-top: 10px; text-align: center; }
    </style>
</head>
<body>

<h1>INFORME GERENCIAL DE COBERTURA ENERGÉTICA</h1>
<p><strong>Período:</strong> {{ periodo }}</p>
<p><strong>Fecha de generación:</strong> {{ fecha_generacion }}</p>

<h2>1. Indicadores principales</h2>
<div class="kpi-grid">
    <div class="kpi-card">
```

```
    # Guardarla como archivo (el enunciado exige el nombre plantilla.html)
    (BASE / 'plantilla.html').write_text(template_informe, encoding='utf-8')
    print("Plantilla guardada: plantilla.html")
```

```
Plantilla guardada: plantilla.html
```

- Exportar el informe a informe.docx o informe.pdf usando las técnicas trabajadas en la Guía 7. Incorporar título, periodo, 3 KPIs, 1 gráfico, 1 hallazgo, 1 recomendación y 1 limitación real.

```
# Renderizar
template = Template((BASE / 'plantilla.html').read_text(encoding='utf-8'))
html_renderizado = template.render(**datos_template)

ruta_html = BASE / 'informe.html'
ruta_html.write_text(html_renderizado, encoding='utf-8')
print(f"HTML generado: {ruta_html.name}")

# convertir a pdf
from weasyprint import HTML as WeasyHTML

ruta_pdf = BASE / 'informe.pdf'
WeasyHTML(string=html_renderizado).write_pdf(str(ruta_pdf))

print(f"PDF generado: {ruta_pdf.name}")
print(f"Tamaño: {ruta_pdf.stat().st_size / 1024:.1f} KB")
```

```
WARNING:weasyprint:Using fontTools instead of HarfBuzz-Subset for font "Liberation
WARNING:weasyprint:Using fontTools instead of HarfBuzz-Subset for font "Liberation
HTML generado: informe.html
PDF generado: informe.pdf
Tamaño: 72.4 KB
```



INFORME GERENCIAL DE COBERTURA ENERGÉTICA

Periodo: 01/01/2026 - 29/06/2026

Fecha de generación: 15/10/2026 19:30

1. Indicadores principales

ENERGÍA GENERADA (KWH)	DEMANDA TOTAL (KWH)	DEFICIT TOTAL (KWH)
2,913,623.55	3,146,866.67	286,904.20

2. Déficit por sede

3. Hallazgo

Las sedes S04 (77,336 kWh) y S07 (69,364 kWh) concentran 51.1% del déficit total del periodo 01/01/2026 - 29/06/2026 (286,904 kWh). Esto describe dónde se ubica el déficit, no su causa.

4. Recomendación

Investigar las sedes S04 y S07, que concentran el 51.1% del déficit de energía, para identificar por qué su generación no alcanza la demanda de estas sedes.

5. Limitación

El análisis del déficit de energía comprende un periodo de 180 días, por lo que únicamente sirve como prototipo, no para tomar decisiones reales. Además, el análisis solo muestra dónde se genera el déficit de energía, no sus posibles causas.

Informe generado automáticamente por generar_reporte.py

- Generar reporte_kpis.csv con indicador,valor para los tres KPIs oficiales y metadatos.json que declare fuente, registros_entrada y sha256_fuente del archivo operacion_sedes.csv.

```
#reporte_kpis.csv con indicador y valor para los kpis
df_kpis = pd.DataFrame({
    'indicador': ['energia_total_kwh', 'demanda_total_kwh', 'deficit_total_kwh'],
    'valor': [round(kpis['energia_total_kwh'], 2),
              round(kpis['demanda_total_kwh'], 2),
              round(kpis['deficit_total_kwh'], 2)]
})
ruta_kpis = BASE / 'reporte_kpis.csv'
df_kpis.to_csv(ruta_kpis, index=False)
print(df_kpis.to_string(index=False))

# metadatos.json: fuente, registros_entrada, sha256_fuente
sha256_fuente = hashlib.sha256(ruta_fuente.read_bytes()).hexdigest()

metadatos = {
    'fuente': 'operacion_sedes.csv',
    'registros_entrada': len(df),
    'sha256_fuente': sha256_fuente,
}
ruta_meta = BASE / 'metadatos.json'
ruta_meta.write_text(json.dumps(metadatos, indent=2, ensure_ascii=False), encoding='utf-8')
print(json.dumps(metadatos, indent=2, ensure_ascii=False))
```

```
        indicador      valor
energia_total_kwh 2913623.55
demanda_total_kwh 3146866.67
deficit_total_kwh  286904.20
{
  "fuente": "operacion_sedes.csv",
  "registros_entrada": 1620,
  "sha256_fuente": "9c28479ba39e4204b20bacdb23341e155ad1b9d8d517338906ae540b005520cb"
}
```

reporte_kpis.csv

1 to 3 of 3 entries Filter

indicador	valor
energia_total_kwh	2913623.55
demanda_total_kwh	3146866.67
deficit_total_kwh	286904.2

Show 10 per page

reporte_kpis.csv metadatos.json


```
{
  "fuente": "operacion_sedes.csv",
  "registros_entrada": 1620,
  "sha256_fuente": "9c28479ba39e4204b20bacdb23341e155ad1b9d8d517338906ae540b0055
}
```


- Demostrar que la salida puede regenerarse ejecutando el mismo script, sin editar manualmente el informe.

  Podemos observar que al ser dos veces ejecutado lo único que cambia es la fecha de generación de cada documento.





Justificación técnica (máximo 5 líneas): describa qué cambia al regenerar el informe, qué dato comprueba su concordancia con el dashboard y cuál es la principal limitación operativa del caso.

Respuesta:

Al generar el informe solo cambia la fecha de generación, pues el script y la plantilla no se modifican manualmente; los KPIS, el grafico, el hallazgo y el sha256_fuente solo variarían si el CSV cambia, manteniendo las mismas columnas. Al compararlo con el dashboard de Power BI, concuerdan deficit_total_kwh (286,904.20), energia_total_kwh (2,913,623.55) y demanda_total_kwh (3,146,866.67), con una cobertura de 90.88%. La principal limitación del caso es que el informe cubre solamente 180 días de datos sintéticos, por lo que nos sirve únicamente como prototipo, y muestra donde se encuentra el déficit de energía, no las razones por las que ocurre.

Parte 4. Fundamentos de predicción con IA — 30 puntos

Problema: Se requiere estimar la demanda energética del siguiente día con un modelo inicial, sin anticipar información futura ni presentar resultados como certezas.

- Abrir demanda_diaria.csv, ordenar cronológicamente, revisar nulos y definir y = demanda_kwh.

modelo _P4.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Compartir

Comandos + Código + Texto Ejecutar todo

RAM Disco

```
[2]
✓ 26 s
# CARGAR demanda_diaria.csv Y ORDENAR POR CRONOLOGIA
if not os.path.exists("demanda_diaria.csv"):
    from google.colab import files
    print("Selecciona el archivo demanda_diaria.csv")
    files.upload()

df = pd.read_csv("demanda_diaria.csv", parse_dates=["fecha"])
df = df.sort_values("fecha").reset_index(drop=True) # Orden cronologico

# Variables de calendario
df["dia_semana"] = df["fecha"].dt.dayofweek  # 0=lunes, 6=domingo
df["mes"] = df["fecha"].dt.month

# Variables de calendario
print(f"Dataset cargado: {df.shape[0]} días, {df.shape[1]} columnas")
print(f"\nPrimeras 5 filas:")
print(df.head().to_string())
```

```
Selecciona el archivo demanda_diaria.csv
 Elegir archivos  Sin archivos seleccionados Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving demanda_diaria.csv to demanda_diaria.csv
Dataset cargado: 180 días, 5 columnas

Primeras 5 filas:
       fecha  demanda_kwh  evento_programado  dia_semana  mes
0 2026-01-01     17856.31                  1           3    1
1 2026-01-02     17379.10                  1           4    1
2 2026-01-03     15720.70                  0           5    1
3 2026-01-04     15694.05                  0           6    1
4 2026-01-05     16505.77                  0           0    1
```

```
[3]
✓ 0 s
# VERIFICAR CALIDAD DEL DATASET
print("INFORMACIÓN DEL DATASET:")
print("=" * 50)
df.info()

print("\nVALORES NULOS POR COLUMNA:")
print(df.isnull().sum())

print(f"\nFECHAS DUPLICADAS: {df['fecha'].duplicated().sum()}")
print(f"¿Fechas en orden cronológico? {df['fecha'].is_monotonic_increasing}")

print(f"\nRANGO TEMPORAL:")
print(f"  Desde: {df['fecha'].min()}")
print(f"  Hasta: {df['fecha'].max()}")

print(f"\nESTADISTICAS DE demanda_kwh (variable objetivo y):")
print(df['demanda_kwh'].describe().to_string())
```

```
INFORMACIÓN DEL DATASET:
==================================================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   fecha              180 non-null    datetime64[ns]
 1   demanda_kwh        180 non-null    float64
 2   evento_programado  180 non-null    int64
 3   dia_semana         180 non-null    int32
 4   mes                180 non-null    int32
dtypes: datetime64[ns](1), float64(1), int32(2), int64(1)
memory usage: 5.8 KB

VALORES NULOS POR COLUMNA:
fecha                0
demanda_kwh          0
evento_programado    0
dia_semana           0
mes                  0
dtype: int64

FECHAS DUPLICADAS: 0
¿Fechas en orden cronológico? True

RANGO TEMPORAL:
  Desde: 2026-01-01 00:00:00
  Hasta: 2026-06-29 00:00:00

ESTADISTICAS DE demanda_kwh (variable objetivo y):
count      180.000000
mean     17482.592611
std       1042.291145
min      15042.510000
25%      16695.587500
50%      17392.615000
75%      18221.675000
max      19847.820000
```

- Crear X con dia_semana, mes, evento_programado, lag_1, lag_7 y media_movil_7. Las tres últimas características se derivan exclusivamente de demanda previa; media_movil_7 debe excluir la demanda del mismo día mediante shift(1).

```
# CREAR VARIABLES DE REZAGO
#.shift(1) Obtiene demanda de ayer y .shift(7) la de hace 7 dias

df["lag_1"] = df["demanda_kwh"].shift(1)
df["lag_7"] = df["demanda_kwh"].shift(7)

print("VARIABLES DE REZAGO:")
print("=" * 70)
print(df[["fecha", "demanda_kwh", "lag_1", "lag_7"]].head(10).to_string())
```

```
# CREAR MEDIA MOVIL SIN FUGA
#.shift(1)  para excluir el dia actual
#.rolling(window=7) para obtener los ultimos 7 dias
df["media_movil_7"] = (
    df["demanda_kwh"]
    .shift(1)
    .rolling(window=7)
    .mean()
)
```

```
#DEFINIR VARIABLES PREDICTORAS (X) Y OBJETIVO (y)

features = [
    "dia_semana",          # Día de la semana (0-6)
    "mes",                 # Mes del año (1-12)
    "evento_programado",   # ¿Hay evento programado? (0/1)
    "lag_1",               # Demanda de ayer
    "lag_7",               # Demanda de hace 7 días
    "media_movil_7"        # Promedio de últimos 7 días (excluyendo hoy)
]

X = df_modelo[features]
y = df_modelo["demanda_kwh"]
```

- Eliminar las filas iniciales sin historial, realizar división cronológica 80/20, sin mezclar aleatoriamente.

[8] ✓ 0s

```
#ELIMINAR FILAS CON NULOS
print(f"Filas ANTES de eliminar NaN: {len(df)}")


df_modelo = df.dropna(subset=["lag_1", "lag_7", "media_movil_7"]).copy()
df_modelo = df_modelo.reset_index(drop=True)

print(f"Filas DESPUÉS de eliminar NaN: {len(df_modelo)}")
print(f"Filas eliminadas: {len(df) - len(df_modelo)}")
print(f"\nLas filas eliminadas son las primeras que no tienen historial de 7 días")

print(f"\nPrimeras 3 filas del dataset de modelado:")
print(df_modelo.head(3).to_string())
```

```
Filas ANTES de eliminar NaN: 180
Filas DESPUÉS de eliminar NaN: 173
Filas eliminadas: 7

Las filas eliminadas son las primeras que no tienen historial de 7 días

Primeras 3 filas del dataset de modelado:
       fecha  demanda_kwh  evento_programado  dia_semana  mes  tendencia_30     lag_1     lag_
0 2026-01-08     16568.10                  0           3    1           NaN  17456.69  17856.3
1 2026-01-09     15495.83                  0           4    1           NaN  16568.10  17379.7
2 2026-01-10     15370.24                  0           5    1           NaN  15495.83  15720.7
```

[10]

```
#SPLIT TEMPORAL 80/20

corte = int(len(df_modelo) * 0.80)

X_train = X.iloc[:corte]
X_test = X.iloc[corte:]

y_train = y.iloc[:corte]
y_test = y.iloc[corte:]

fecha_test = df_modelo["fecha"].iloc[corte:]

print("SPLIT TEMPORAL 80/20")
print("=" * 60)
print(f"Total de registros: {len(df_modelo)}")
print(f"Entrenamiento: {len(X_train)} registros (80%)")
print(f"Prueba: {len(X_test)} registros (20%)")

print(f"\nRango de ENTRENAMIENTO:")
print(f"   Desde: {df_modelo['fecha'].iloc[0]}")
print(f"   Hasta: {df_modelo['fecha'].iloc[corte-1]}")

print(f"\nRango de PRUEBA:")
print(f"   Desde: {df_modelo['fecha'].iloc[corte]}")
print(f"   Hasta: {df_modelo['fecha'].iloc[-1]}")

# Verificar que no hay solapamiento
print(f"\nVERIFICACIÓN:")
print(f"   Última fecha entrenamiento: {df_modelo['fecha'].iloc[corte-1]}")
print(f"   Primera fecha prueba: {df_modelo['fecha'].iloc[corte]}")
print(f"   ¿Prueba después de entrenamiento? {df_modelo['fecha'].iloc[corte] > df_modelo['fe
```

```
SPLIT TEMPORAL 80/20
============================================================
Total de registros: 173
Entrenamiento: 138 registros (80%)
Prueba: 35 registros (20%)

Rango de ENTRENAMIENTO:
   Desde: 2026-01-08 00:00:00
   Hasta: 2026-05-25 00:00:00

Rango de PRUEBA:
   Desde: 2026-05-26 00:00:00
   Hasta: 2026-06-29 00:00:00

VERIFICACIÓN:
   Última fecha entrenamiento: 2026-05-25 00:00:00
   Primera fecha prueba: 2026-05-26 00:00:00
   ¿Prueba después de entrenamiento? True
```

- Entrenar DummyRegressor(strategy="mean") como baseline y LinearRegression() como modelo principal (Scikit-Learn).

[11] ✓ 0s

```
# CONSTRUIR BASELINE
# Estrategia: predecir siempre la media del conjunto de entrenamiento

baseline = DummyRegressor(strategy="mean")
baseline.fit(X_train, y_train)

pred_baseline = baseline.predict(X_test)

# Calcular métricas del baseline
mae_base = mean_absolute_error(y_test, pred_baseline)
rmse_base = mean_squared_error(y_test, pred_baseline) ** 0.5
r2_base = r2_score(y_test, pred_baseline)

print("BASELINE")
print("=" * 60)
print(f"Media del entrenamiento: {y_train.mean():,.2f} kWh")
print(f"\nMÉTRICAS DEL BASELINE:")
print(f"   MAE:  {mae_base:,.2f} kWh")
print(f"   RMSE: {rmse_base:,.2f} kWh")
print(f"   R²:   {r2_base:.4f}")
```

```
BASELINE
============================================================
Media del entrenamiento: 17,281.82 kWh

MÉTRICAS DEL BASELINE:
   MAE:  1,194.57 kWh
   RMSE: 1,391.68 kWh
   R²:   -1.9384
```

[12] ✓ 0s

```
#ENTRENAR REGRESIÓN LINEAL

modelo = LinearRegression()
modelo.fit(X_train, y_train)

# Predecir sobre el conjunto de prueba
pred = modelo.predict(X_test)

# Calcular métricas
mae = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred) ** 0.5
r2 = r2_score(y_test, pred)

print("REGRESIÓN LINEAL")
print("=" * 60)
print(f"\nMÉTRICAS DEL MODELO:")
print(f"   MAE:  {mae:,.2f} kWh")
print(f"   RMSE: {rmse:,.2f} kWh")
print(f"   R²:   {r2:.4f}")
```

```
REGRESIÓN LINEAL
============================================================

MÉTRICAS DEL MODELO:
   MAE:  298.77 kWh
   RMSE: 378.72 kWh
   R²:   0.7824
```

- Exportar predicciones.csv con columnas fecha,real,baseline,prediccion, en orden temporal para el conjunto de prueba, y metricas.json con MAE del baseline, MAE y RMSE del modelo, R^2 y porcentaje de mejora del MAE.

[18] ✓ 0 s

```python
# GUARDAR RESULTADOS: predicciones.csv y metricas.json

# predicciones.csv
predicciones = pd.DataFrame({
    "fecha": fecha_test.dt.strftime("%Y-%m-%d").values,
    "real": y_test.values,
    "baseline": pred_baseline,
    "prediccion": pred
}).sort_values("fecha").reset_index(drop=True)

predicciones.to_csv("predicciones.csv", index=False, encoding="utf-8")
print(f"Predicciones guardadas: predicciones.csv ({len(predicciones)} filas)")

# metricas.json
metricas = {
    "mae_baseline": round(float(mae_base), 4),
    "mae_modelo": round(float(mae), 4),
    "rmse_modelo": round(float(rmse), 4),
    "r2_modelo": round(float(r2), 4),
    "mejora_mae_pct": round(float(mejora_mae), 2)
}
with open("metricas.json", "w", encoding="utf-8") as f:
    json.dump(metricas, f, indent=2)
print("Métricas guardadas: metricas.json")

print("\nCONTENIDO DE metricas.json:")
print(json.dumps(metricas, indent=2))

print("\nPRIMERAS 5 FILAS DE predicciones.csv:")
print(predicciones.head().to_string(index=False))

# Descargar los archivos
from google.colab import files
files.download("predicciones.csv")
files.download("metricas.json")
print("\nArchivos descargados")
```

```
Predicciones guardadas: predicciones.csv (35 filas)
Métricas guardadas: metricas.json

CONTENIDO DE metricas.json:
{
  "mae_baseline": 1194.573,
  "mae_modelo": 298.765,
  "rmse_modelo": 378.7246,
  "r2_modelo": 0.7824,
  "mejora_mae_pct": 74.99
}

PRIMERAS 5 FILAS DE predicciones.csv:
     fecha     real     baseline   prediccion
2026-05-26 18800.00 17281.821304 18603.319450
2026-05-27 19294.76 17281.821304 18371.300370
2026-05-28 18761.74 17281.821304 18254.034120
2026-05-29 17890.43 17281.821304 17746.713486
2026-05-30 17112.40 17281.821304 17218.995397

Archivos descargados
```

- Guardar el notebook ejecutable modelo.ipynb y explicar si el modelo realmente agrega valor frente al baseline.

```
#INTERPRETACIÓN
primera_fecha = fecha_test.iloc[0].strftime("%Y-%m-%d")
ultima_fecha = fecha_test.iloc[-1].strftime("%Y-%m-%d")
veredicto = "Si agrega valor" if mae < mae_base else "No agrega valor"

print("¿EL MODELO AGREGA VALOR FRENTE AL BASELINE?")
print("=" * 70)
print(f"   {veredicto}: MAE {mae_base:,.2f} -> {mae:,.2f} kWh ({mejora_mae:.2f}% mejor), R² = {r2:.2f}")
print(f"   el error medio sigue siendo ~{mae:,.0f} kWh, es una estimación, no una certeza.")

print(f"   Periodo de prueba: {primera_fecha} al {ultima_fecha}")
```

```
¿EL MODELO AGREGA VALOR FRENTE AL BASELINE?
======================================================================
   Si agrega valor: MAE 1,194.57 -> 298.77 kWh (74.99% mejor), R² = 0.78
   el error medio sigue siendo ~299 kWh, es una estimación, no una certeza.
   Periodo de prueba: 2026-05-26 al 2026-06-29
```

Justificación técnica (máximo 5 líneas): cite la primera y última fecha de prueba, ambos MAE y la mejora %. Explique por qué no debe entrenarse usando el futuro y qué condiciones limitan el uso real del pronóstico.

Respuesta:

En la prueba realizada del 2026-05-26 al 2026-06-29 el MAE del baseline fue 1.194,57 kWh y el obtenido por la regresión lineal 298,77 kWh obteniendo una mejora del 74,99 %, así que el modelo sí agrega valor. No debe entrenarse con el futuro porque sería una fuga de información: usaría datos que no existen al pronosticar e inflaría las métricas; por eso la división es cronológica y media_movil_7 usa shift(1). El uso real exige contar con la demanda de ayer, la de hace 7 días y el evento programado, y solo se validó en 35 días con un modelo lineal. El error medio sigue siendo de unos 300 kWh, por ende, solo es una estimación, pero no es una certeza.

7. Contrato de salida para revisión semiautomatizada

Estos nombres, columnas y claves son obligatorios. La revisión automática no ejecutará código estudiantil: comparará los productos contra cálculos independientes realizados sobre los datasets oficiales.

Archivo	Estructura exacta
indicadores.csv	indicador,valor | energia_total_kwh; demanda_total_kwh; deficit_total_kwh; cobertura_pct
geo_validado.csv	sede_id,latitud,longitud,deficit_total_kwh
reporte_kpis.csv	indicador,valor | energia_total_kwh; demanda_total_kwh; deficit_total_kwh
metadatos.json	fuente, registros_entrada, sha256_fuente
predicciones.csv	fecha,real,baseline,prediccion | Fecha AAAA-MM-DD y decimales con punto
metricas.json	mae_baseline, mae_modelo, rmse_modelo, r2_modelo, mejora_mae_pct
equipo.json	{"equipo":"E01","seccion":"IDS221"} (adapte código y sección)
integrantes.csv	carnet,nombre,aporte | 4 o 5 integrantes del equipo

8. Entregables y medio de entrega

Suban UN archivo EP2_EquipoXX.zip al buzón Moodle. El archivo debe contener, como mínimo, los siguientes elementos (se aceptan subcarpetas; no cambien los nombres de los archivos). Este mismo entregable deberá estar en github:

- Identificación: equipo.json, integrantes.csv y justificaciones.md (4 justificaciones identificadas P1, P2, P3 y P4).
- Power BI: dashboard.pbix, vista_bi.png e indicadores.csv.
- ArcGIS: geo_validado.csv, mapa_vista1.png, mapa_vista2.png; enlace de ArcGIS dentro de justificaciones.md cuando el acceso lo permita.
- Reportería: generar_reporte.py, plantilla.html, informe.html, informe.docx o informe.pdf, grafico.png, reporte_kpis.csv y metadatos.json.

- Predicción: modelo.ipynb, predicciones.csv y metricas.json.

REGLA DE EVIDENCIA: Una captura no sustituye el archivo funcional. Un dato correcto copiado en un CSV sin evidencia de proceso no demuestra por sí solo autoría; el docente podrá pedir reproducir un filtro, ejecutar un script o explicar una predicción. El vídeo demostrativo reforzará el entregable

9. Rúbrica de evaluación por competencias

Competencia / parte	Verificación automática	Validación docente	Total
1. Power BI	15: cuatro KPIs reproducidos	10: PBIX, medidas, visuales, filtro y claridad	25
2. ArcGIS	10: sede/posición/déficit válidos	10: dos vistas, estilos, pop-ups, patrones	20
3. Reportería	17: integridad, KPIs, archivos, metadata	3: regeneración y coherencia	20
4. Predicción	25: fechas, baseline, regresión y métricas	5: interpretación y ausencia de fuga	30
Comprobación individual	—	**5: explicación o cambio en vivo por integrante**	5
TOTAL	67 puntos	33 puntos	100

10. Criterios generales y comprobación individual

- Funcionalidad y exactitud: todos los resultados deberán corresponder a los archivos oficiales sin alterar entradas para mejorar artificialmente métricas.
- Reproducibilidad: el informe y el notebook deben ejecutarse; cuando haya fallos, se valorará el intento documentado y la evidencia verificable.
- Integridad y colaboración: cada integrante registra su aporte en integrantes.csv y explica una decisión, fórmula, mapa o métrica propia.
- Comprobación breve: el docente podrá solicitar un cambio de filtro en vivo o una explicación concreta sobre el resultado generado. **El componente individual puede producir notas distintas dentro del equipo.**
- Retroalimentación: el validador identificará criterios superados y pendientes; los puntos visuales, interpretación y autoría los registrará el docente.

Éxito en la evaluación. Prioricen una solución que funciona, se puede comprobar y pueden explicar, antes que un producto visualmente complejo sin evidencia técnica.